# BUG REPORT

BUG 1 - RTL Drain-Cycle Timing (controller.sv)

Activation: make all BUG1=1

Location of injection. The FSM controller defines a parameter DRAIN_CYCLES, which determines how many cycles the controller waits after the last input is fed into the systolic array before asserting result_valid. The correct value is 2*N cycles, accounting for the full pipeline depth of the array.

Nature of the bug. The value is reduced by one cycle, from 2*N to 2*N - 1.

Expected failure mode. Because the systolic array is a pipeline in which data propagates from each processing element to the next, asserting result_valid one cycle early causes the final processing element PE[N-1][N-1] to be sampled before its last multiply-accumulate has completed. The result is a single incorrect element at position C[N-1][N-1], while all other elements remain correct.

Observation. The scoreboard reports a UVM_ERROR localized to the last element of the output matrix.

**How to debug:**

1. Open Verdi waveform - search for result_valid signal
2. Count clock cycles from last valid_in going low to result_valid going high
3. You will see it fires at cycle 2*N-1 instead of 2*N
4. Go to controller.sv - check DRAIN_CYCLES parameter value
5. Fix: change DRAIN_CYCLES = 2*N - 1 back to DRAIN_CYCLES = 2*N

BUG 2 - RTL Accumulator Reset (pe.sv)

Activation: make all BUG2=1

Location of injection. Each Processing Element (PE) contains a clear signal driven by the controller at the start of every new transaction, resetting the accumulator to zero before fresh operands arrive.

Nature of the bug. The clear branch is removed from the PE; the accumulator no longer responds to the clear signal.

Expected failure mode. The first transaction completes correctly because all accumulators are zero following the global reset. Beginning with the second transaction, however, residual values from the previous matrix multiply remain in the PEs, and new partial products are added on top of stale state. This causes systematic accumulation drift across transactions.

Observation. The first transaction passes cleanly. From the second transaction onward, the scoreboard issues a UVM_ERROR for every element, and the error count grows

monotonically with each transaction. This is among the most visible failure patterns in the log.

**How to debug:**

1. Open Verdi - find accum signal inside any PE instance
2. Watch accum across two back-to-back transactions
3. You will see accum never goes to zero between transactions - it keeps growing
4. Go to pe.sv - look at the always_ff block
5. Check if if (clear) accum <= '0 branch exists
6. Fix: restore the clear priority branch above the valid_in branch

BUG 3 - UVM Monitor Index Swap (mm_monitor.sv)

Activation: make all BUG3=1

Location of injection. The monitor samples the flat result bus on result_valid and converts it into a two-dimensional matrix got_C via a function unpack_result. The correct mapping stores the bus element at position [r][c] into got_C[r][c].

Nature of the bug. The indices are swapped: bus element [r][c] is stored into got_C[c][r]. The captured matrix is therefore the transpose of the actual DUT output.

Expected failure mode. The RTL continues to compute A × B correctly, and the reference model correctly predicts A × B. The monitor, however, reports (A × B)^T to the scoreboard, which then compares the correct expected matrix against the transposed observed matrix. Mismatches occur for all off-diagonal elements C[r][c] where r != c. Symmetric output matrices, by construction, would not expose this bug, which is itself a useful demonstration of why coverage of non-symmetric inputs matters.

Observation. UVM_ERROR fires for every off-diagonal element of every transaction. Waveform inspection of the result bus confirms that the hardware output is correct, isolating the fault to the monitor's data interpretation rather than to the RTL.

**How to debug:**

1. In sim log - find a failing transaction and note exp and got values
2. Observe that got_C[r][c] matches exp_C[c][r] - transpose relationship
3. Open Verdi - check result bus values directly - they are correct
4. Since DUT is correct but got_C is wrong - bug is in monitor unpacking
5. Go to mm_monitor.sv -> unpack_result function
6. Fix: change res_mat[c][r] back to res_mat[r][c]

BUG4 - UVM Reference Model Corruption (mm_seq_item.sv, mm_sequence.sv, mm_test.sv)

Activation: make all BUG4=1

Location of injection. The reference model is implemented as compute_expected() inside mm_seq_item.sv, and is invoked by both the sequence (to predict expected results) and the

monitor (during cross-checking). A second component of this bug introduces a new sequence class, mm_error_seq, which is substituted for mm_test_seq during Phase B of the test under the `ifdef BUG4 guard in mm_test.sv.

Nature of the bug. The dot-product operation in compute_expected() is replaced with element-wise addition:

Correct: C[r][c] += A[r][k] * B[k][c]
Injected: C[r][c] += A[r][k] + B[k][c]

Expected failure mode. The DUT continues to compute matrix multiplication correctly. The reference model, however, now produces element-wise sums rather than dot products, so the expected matrix delivered to the scoreboard is systematically incorrect. The scoreboard compares incorrect expected values against correct DUT output, producing UVM_ERROR for virtually every non-zero element.

Observation. Almost every element of every Phase B transaction (all 16 random tests) is flagged as a mismatch. The bug is traceable to two specific points in the codebase: the mm_error_seq class in mm_sequence.sv, and the `ifdef BUG4 block within compute_expected() in mm_seq_item.sv.

**How to debug:**

1. In sim log - compare exp and got values for a failing element
2. Manually calculate: if exp = A[r][0]+B[0][c] + A[r][1]+B[1][c]... -> addition pattern confirmed
3. Correct answer matches got - DUT is right, expected is wrong
4. Since expected comes from compute_expected() - go to mm_seq_item.sv
5. Find the inner loop - check if * multiplication operator is replaced with +
6. Fix: change A[r][k] + B[k][c] back to A[r][k] * B[k][c]